

Vous avez les tables et données suivantes :

```
DROP TABLE IF EXISTS utilisateurs CASCADE;
CREATE TABLE utilisateurs (
  id INTEGER PRIMARY KEY,
  pseudo VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  date_inscription DATE DEFAULT CURRENT_DATE,
  parrain_id INTEGER REFERENCES utilisateurs(id)
);
```

```
DROP TABLE IF EXISTS jeux CASCADE;
CREATE TABLE jeux (
  id INTEGER PRIMARY KEY,
  titre VARCHAR(100) NOT NULL,
  editeur VARCHAR(50),
  annee_sortie INTEGER,
  prix DECIMAL(5,2)
);
```

```
DROP TABLE IF EXISTS bibliotheques CASCADE;
CREATE TABLE bibliotheques (
  id INTEGER PRIMARY KEY,
  utilisateur_id INTEGER NOT NULL REFERENCES utilisateurs(id),
  jeu_id INTEGER NOT NULL REFERENCES jeux(id),
  date_achat DATE,
  heures_jouees DECIMAL(6,1)
);
```

```
DROP TABLE IF EXISTS avis CASCADE;
CREATE TABLE avis (
  id INTEGER PRIMARY KEY,
  utilisateur_id INTEGER NOT NULL REFERENCES utilisateurs(id),
  jeu_id INTEGER NOT NULL REFERENCES jeux(id),
  note INTEGER CHECK (note BETWEEN 1 AND 5),
  commentaire TEXT,
  date_avis DATE
);
```

```
INSERT INTO utilisateurs VALUES
(1, 'ShadowSlayer', 'shadow@email.com', '2023-01-15', NULL),
(2, 'PixelQueen', 'pixel@email.com', '2023-03-22', 1),
(3, 'NoobMaster42', 'noob@email.com', '2023-06-10', 1),
(4, 'DragonFury', 'dragon@email.com', '2024-01-05', 2),
```

```
(5, 'LootGoblin', 'loot@email.com', '2024-02-14', 2),
(6, 'StealthArcher', 'stealth@email.com', '2024-05-30', 3),
(7, 'MageFireball', 'mage@email.com', '2024-08-18', NULL),
(8, 'TankBuster', 'tank@email.com', '2025-01-10', 7),
(9, 'NoobBuster', 'buster@email.com', '2024-09-19', NULL);
```

INSERT INTO jeux VALUES

```
(1, 'Elden Ring', 'Bandai Namco', 2022, 69.99),
(2, 'Baldur's Gate 3', 'Larian Studios', 2023, 59.99),
(3, 'Cyberpunk 2077', 'CD Projekt Red', 2020, 29.99),
(4, 'Hollow Knight', 'Team Cherry', 2017, 14.99),
(5, 'Stardew Valley', 'ConcernedApe', 2016, 14.99),
(6, 'God of War Ragnarok', 'Sony', 2022, 69.99),
(7, 'Portal 2', 'Valve', 2011, 9.99),
(8, 'Among Us', 'Innersloth', 2018, 4.99),
(9, 'Witcher 3', 'CD Projekt Red', 2015, 29.99);
```

INSERT INTO bibliotheques VALUES

```
(1, 1, 1, '2023-01-20', 120.5),
(2, 1, 3, '2023-02-10', 45.0),
(3, 1, 7, '2023-03-05', 8.5),
(4, 2, 2, '2023-04-01', 200.0),
(5, 2, 5, '2023-04-15', 350.0),
(6, 2, 6, '2024-01-20', 60.0),
(7, 3, 1, '2023-07-01', 15.0),
(8, 3, 8, '2023-07-15', 500.0),
(9, 4, 2, '2024-01-20', 180.0),
(10, 4, 4, '2024-02-10', 40.0),
(11, 5, 3, '2024-03-01', 75.0),
(12, 5, 5, '2024-03-15', 20.0),
(13, 6, 1, '2024-06-10', 95.0),
(14, 6, 4, '2024-06-20', 55.0),
(15, 6, 7, '2024-07-01', 30.0),
(16, 7, 2, '2024-09-01', 250.0),
(17, 7, 6, '2024-09-15', 10.0),
(18, 8, 1, '2025-01-15', 5.0),
(19, 8, 8, '2025-01-20', 12.0);
```

INSERT INTO avis VALUES

```
(1, 1, 1, 5, 'Chef-d"oeuvre absolu !', '2023-02-01'),
(2, 1, 3, 3, 'Bonne ambiance mais buggé au lancement', '2023-03-15'),
(3, 2, 2, 5, 'Jeu de rôle ultime, 200h et pas lassé', '2023-05-01'),
(4, 2, 5, 4, 'Parfait pour se détendre', '2023-06-01'),
(5, 3, 1, 2, 'Trop difficile pour moi...', '2023-08-01'),
```

(6, 3, 8, 4, 'Excellent pour les soirées entre amis', '2023-09-01'),
(7, 4, 2, 5, 'Meilleur jeu de rôle jamais créé', '2024-03-01'),
(8, 4, 4, 5, 'Un bijou animation et de gameplay', '2024-03-15'),
(9, 5, 3, 4, 'Beaucoup amélioré depuis le lancement', '2024-04-01'),
(10, 6, 1, 5, 'Difficile mais gratifiant', '2024-07-01'),
(11, 6, 7, 5, 'Un des meilleurs puzzles games', '2024-08-01'),
(12, 7, 2, 4, 'Excellent mais une fin un peu rapide', '2024-10-01'),
(13, 8, 1, 3, 'Je débute, ça va venir...', '2025-02-01');

Répondez aux questions suivantes :

Question 1 : Afficher tous les noms de jeux, les commentaires et les notes laissés. Les jeux sans avis doivent apparaître aussi.

```
SELECT j.titre, a.commentaire, a.note  
FROM jeux AS j  
LEFT JOIN avis AS a ON j.id = a.jeu_id;
```

Question 2 : Afficher le pseudo de chaque utilisateur qui possède au moins un jeu.

```
SELECT DISTINCT u.pseudo  
FROM utilisateurs u  
JOIN bibliotheques b ON u.id = b.utilisateur_id;
```

Question 3 : Trouver les jeux qui n'ont reçu aucun avis.

```
SELECT j.titre  
FROM jeux j  
LEFT JOIN avis a ON j.id = a.jeu_id  
WHERE a.id IS NULL;  
WHERE a.jeu_id IS NULL;
```

Question 4 : Trouver les utilisateurs qui n'ont aucun jeu dans leur bibliothèque.

```
SELECT u.pseudo  
FROM utilisateurs u  
LEFT JOIN bibliotheques b ON u.id = b.utilisateur_id  
WHERE b.id IS NULL;  
SELECT u.pseudo  
FROM bibliotheques AS b  
RIGHT JOIN utilisateurs AS u ON b.utilisateur_id = u.id  
WHERE b.id IS NULL;
```

Question 5 : Afficher les nombres moyens d'heures jouées pour chaque jeu.

```
SELECT j.titre, AVG(b.heures_jouees) AS moyenne_heure  
FROM jeux j  
JOIN bibliotheques b ON j.id = b.jeu_id  
GROUP BY j.id, j.titre;
```

Question 6 : Afficher le pseudo de chaque utilisateur et le titre des jeux qu'il possède dans sa bibliothèque.

```
SELECT u.pseudo, j.titre
FROM utilisateurs u
JOIN bibliotheques b ON u.id = b.utilisateur_id
JOIN jeux j ON b.jeu_id = j.id;
SELECT u.pseudo, j.titre
FROM bibliotheques b
JOIN utilisateurs u ON u.id = b.utilisateur_id
JOIN jeux j ON b.jeu_id = j.id;
```

-- Ne marche pas

```
SELECT u.pseudo, j.titre
FROM utilisateurs u
JOIN jeux j ON bibliotheques.jeu_id = j.id
JOIN bibliotheques b ON u.id = b.utilisateur_id;
```

Question 7 : Afficher les pseudos, titres de jeux et notes pour tous les avis laissés.

```
SELECT u.pseudo, j.titre, a.note
FROM avis a
INNER JOIN utilisateurs u ON u.id = a.utilisateur_id
INNER JOIN jeux j ON j.id = a.jeu_id;
```

Question 8 : Afficher les pseudos, titres et heures jouées, triés par heures jouées décroissantes, limité aux 5 premiers.

```
SELECT u.pseudo, j.titre, b.heures_jouees
FROM utilisateurs u
JOIN bibliotheques b ON u.id = b.utilisateur_id
JOIN jeux j ON j.id = b.jeu_id
ORDER BY b.heures_jouees DESC
LIMIT 5;
```

Question 9 : Pour chaque utilisateur, afficher son pseudo et le nombre de jeux qu'il possède, trié du plus grand au plus petit.

```
SELECT u.pseudo, COUNT(b.jeu_id) AS nombre_jeux
FROM utilisateurs AS u
LEFT JOIN bibliotheques b ON u.id = b.utilisateur_id
GROUP BY u.pseudo, u.id
ORDER BY nombre_jeux DESC;
```

Question 10 : Afficher les pseudos, titres de jeux et notes pour les avis dont la note est supérieure ou égale à 4, trié par note décroissante.

```
SELECT u.pseudo, j.titre, a.note
FROM avis AS a
JOIN utilisateurs AS u ON a.utilisateur_id = u.id
JOIN jeux AS j ON a.jeu_id = j.id
WHERE a.note >= 4
ORDER BY a.note DESC;
```

Question 11 : Afficher tous les utilisateurs et le titre des jeux qu'ils possèdent. Les utilisateurs sans jeu doivent apparaître aussi.

```
SELECT u.pseudo, j.titre
FROM utilisateurs u
LEFT JOIN bibliotheques b ON b.utilisateur_id = u.id
LEFT JOIN jeux j ON b.jeu_id = j.id;
```

Question 12 : Pour chaque utilisateur, afficher son pseudo et la moyenne de ses notes données. Les utilisateurs qui n'ont donné aucun avis doivent apparaître avec NULL.

```
SELECT u.pseudo, AVG(a.note)
FROM utilisateurs AS u
LEFT JOIN avis AS a ON a.utilisateur_id = u.id
GROUP BY u.id, u.pseudo;
```

Question 13 : Afficher tous les jeux et les pseudos des utilisateurs qui les possèdent. L'objectif est que tous les jeux apparaissent, même ceux qui ne sont dans aucune bibliothèque.

```
SELECT j.titre, u.pseudo
FROM jeux j
LEFT JOIN bibliotheques b ON j.id = b.jeu_id
LEFT JOIN utilisateurs u ON b.utilisateur_id = u.id;
```

Question 14 : Afficher tous les utilisateurs et tous les jeux, avec les heures jouées quand un utilisateur possède un jeu.

```
SELECT u.pseudo, j.titre, b.heures_jouees
FROM utilisateurs u
LEFT JOIN bibliotheques b ON u.id = b.utilisateur_id
FULL JOIN jeux AS j ON b.jeu_id = j.id;
```

Question 15 : Trouver les utilisateurs sans jeu et les jeux sans propriétaire.

```
SELECT u.pseudo, j.titre
FROM utilisateurs u
FULL OUTER JOIN bibliotheques b ON u.id = b.utilisateur_id
FULL OUTER JOIN jeux j ON j.id = b.jeu_id
WHERE b.id IS NULL;
```

Question 16 : Pour chaque jeu, afficher son titre et la note moyenne reçue. Tous les jeux doivent apparaître, même ceux sans note.

```
SELECT j.titre, AVG(a.note) AS note_moyenne
FROM jeux j
LEFT JOIN avis a ON a.jeu_id = j.id
GROUP BY j.titre, j.id;
```

Question 17 : Afficher le pseudo de chaque utilisateur et le pseudo de son parrain (celui qui l'a invité).

```
SELECT u.pseudo AS utilisateur, p.pseudo AS parrain
FROM utilisateurs u
LEFT JOIN utilisateurs p ON u.parrain_id = p.id;
```

Question 18 : Trouver les utilisateurs qui ont parrainé au moins 2 personnes, avec le nombre de filleuls.

```
SELECT p.pseudo, COUNT(u.id) AS nb_filleuls
FROM utilisateurs u
JOIN utilisateurs p ON u.parrain_id = p.id
GROUP BY p.pseudo
HAVING COUNT(u.id) >= 2;
```

Question 19 : Trouver les utilisateurs qui n'ont pas été parrainés et qui n'ont parrainé personne.

```
SELECT u.pseudo
FROM utilisateurs u
LEFT JOIN utilisateurs f ON f.parrain_id = u.id
WHERE u.parrain_id IS NULL AND f.id IS NULL;
```

Question 20 : Générer toutes les combinaisons possibles entre les utilisateurs et les jeux.

Afficher seulement les 10 premières lignes.

```
SELECT u.pseudo, j.titre
FROM utilisateurs u
CROSS JOIN jeux j
LIMIT 10;
```

Question 21 : Afficher le commentaire, le pseudo, le titre du jeu, la note donnée et les heures jouées pour chacun des avis.

```
SELECT a.commentaire, u.pseudo, j.titre, a.note, b.heures_jouees
FROM utilisateurs AS u
JOIN avis AS a ON u.id = a.utilisateur_id
JOIN jeux j ON j.id = a.jeu_id
JOIN bibliotheques AS b ON b.jeu_id = j.id AND u.id = b.utilisateur_id;
```

```
SELECT a.commentaire, u.pseudo, j.titre, a.note, b.heures_jouees
FROM avis AS a
LEFT JOIN utilisateurs AS u ON u.id = a.utilisateur_id
LEFT JOIN jeux j ON j.id = a.jeu_id
LEFT JOIN bibliotheques AS b ON (b.jeu_id, b.utilisateur_id) = (a.jeu_id, a.id);
```

Question 22 : Afficher tous les utilisateurs, les jeux qu'ils possèdent (avec heures jouées), et les notes qu'ils ont donné. Inclure les utilisateurs sans jeu et les jeux sans avis.

```
SELECT u.pseudo, j.titre, b.heures_jouees, a.note
FROM utilisateurs u
LEFT JOIN bibliotheques b ON u.id = b.utilisateur_id
FULL JOIN jeux j ON b.jeu_id = j.id
LEFT JOIN avis a ON u.id = a.utilisateur_id AND b.jeu_id = a.jeu_id;
```

```
SELECT u.pseudo, j.titre, b.heures_jouees, a.note
FROM utilisateurs u
LEFT JOIN bibliotheques b ON u.id = b.utilisateur_id
FULL JOIN jeux j ON b.jeu_id = j.id
LEFT JOIN avis a ON b.utilisateur_id = a.utilisateur_id AND b.jeu_id = a.jeu_id;
```

Question 23 : Afficher les jeux du plus rentable au moins rentable en heures jouées moyennes par euro payé. Afficher le nom du jeu et le ratio demandé.

```
SELECT
    j.titre,
    AVG(b.heures_jouees) / j.prix AS ratio
FROM jeux j
JOIN bibliotheques b ON b.jeu_id = j.id
GROUP BY j.id, j.titre, j.prix
ORDER BY ratio DESC;
```

Question 24 : Afficher, pour l'utilisateur ShadowSlayer, toute la chaîne de générations de ses filleuls et leurs propres filleuls.

```
SELECT p.pseudo AS parrain, f1.pseudo AS filleul, f2.pseudo AS petit_filleul
FROM utilisateurs p
JOIN utilisateurs f1 ON f1.parrain_id = p.id
LEFT JOIN utilisateurs f2 ON f2.parrain_id = f1.id
WHERE p.pseudo = 'ShadowSlayer';
```